

Cấu trúc dữ liệu và giải thuật

Bài 2: Đệ quy

GV: Nguyễn Hữu Phát

Nội dung

- Hàm đệ quy và bộ nhớ stack
- Phân tích giải thuật đệ quy
- Nguyên lý đệ quy
- Ưu điểm và khuyết điểm của đệ quy

Giới thiệu khái niệm đệ quy

- Đệ quy là một kỹ thuật lập trình khá hữu dụng trong nhiều trường hợp.
- Xét về bản chất, đệ quy chính là những vòng lặp nhưng dĩ nhiên đệ quy sẽ có những ưu điểm so với vòng lặp thông thường và khi sử dụng đệ quy sẽ đem lại kết quả tối ưu hơn với những cách thực hiện đơn giản hơn.

Ví dụ bài toán đệ quy

Cho $S(n) = 1 + 2 + 3 + \dots + n$
 $\Rightarrow S(10)? S(11)?$

$$S(10) = 1 + 2 + \dots + 10 = 55$$

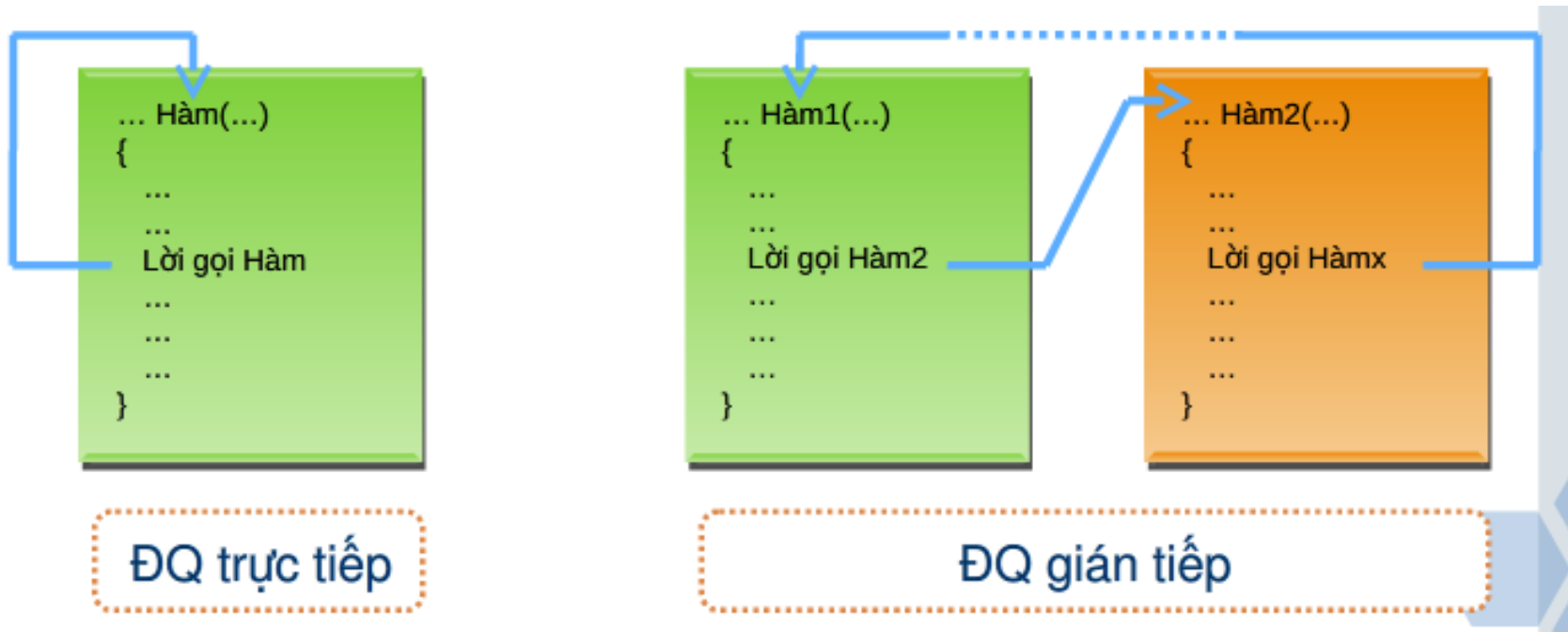
$$\begin{aligned} S(11) &= 1 + 2 + \dots + 10 + 11 = 66 \\ &= S(10) + 11 \\ &= 55 + 11 = 66 \end{aligned}$$

Định nghĩa đệ quy

- Vấn đề đệ quy là vấn đề được định nghĩa bằng chính nó.
- Ví dụ: Tổng $S(n)$ được tính thông qua tổng $S(n-1)$.
- Hai điều kiện quan trọng để hình thành vấn đề đệ quy
 - Điều kiện dừng
 - Tồn tại bước đệ quy

Hàm đệ quy

- Một hàm được gọi là hàm đệ quy nếu bên trong thân của hàm đó có lời gọi hàm lại chính nó một cách trực tiếp hay gián tiếp.



Xây dựng hàm đệ quy

- Việc thiết kế hàm đệ quy chính là quá trình đưa bài toán lớn dần trở thành bài toán nhỏ hơn
- Bài toán cứ vậy nhỏ dần đến mức mà nó đơn giản nhất có thể đưa ra lời giải và quay ngược trở lên, lấy lời giải tìm được để giải bài toán lớn hơn.
- Khi quá trình quay ngược kết thúc đồng nghĩa với việc bài toán lớn nhất được giải quyết.

Phân loại đệ quy

- Xét theo cách thức tự gọi lại hàm, có thể chia đệ quy thành 4 dạng sau:
 - Đệ quy tuyến tính
 - Đệ quy nhị phân
 - Đệ quy hồi tưởng
 - Đệ quy phi tuyến

Đệ quy tuyến tính

- Trong thân hàm có duy nhất một lời gọi hàm gọi lại chính nó một cách tường minh.

```
#include <stdio.h>
int giaithua(int n)
{
    if (n<=2)
    {
        if (n==0)
            return 1;
        return n;
    }
    return n * giaithua(n-1);
}
```

```
int main(){
    int k=3;
    printf("\n Giai thua cua %d la %d",
k,giaithua(k));
    k=1;
    printf("\n Giai thua cua %d la %d",
k,giaithua(k));
    k=2;
    printf("\n Giai thua cua %d la %d",
k,giaithua(k));
}
```

Ví dụ đệ quy tuyến tính

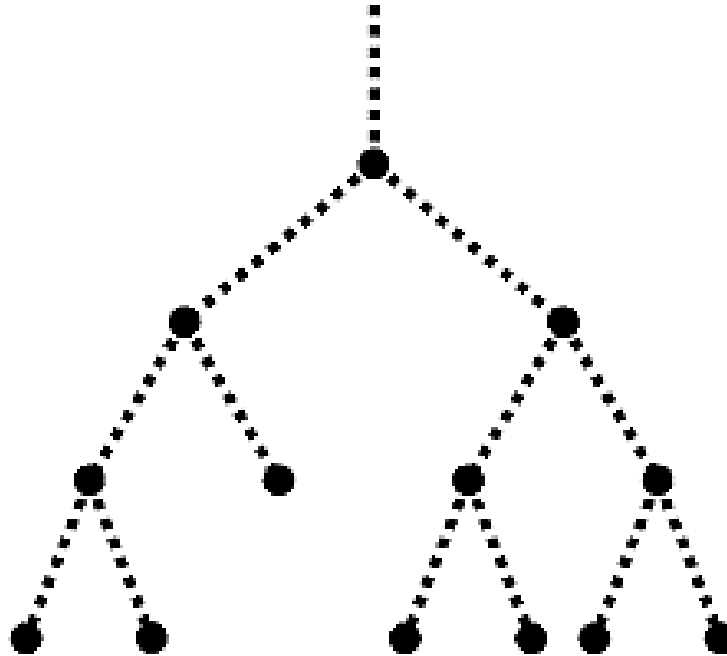
- Ví dụ 1 : Lập hàm tính $n!$ bằng đệ quy

```
int GT(int n)  
{  
    if (n==0)        // điểm dừng  
        return 1;  
    else  
        return n*GT(n-1);  
}
```

$$n! = \begin{cases} n * (n - 1)! \\ 0! = 1 \end{cases}$$

Đệ quy nhị phân

- Trong thân hàm có **hai lời gọi hàm** gọi lại chính nó một cách tường minh.



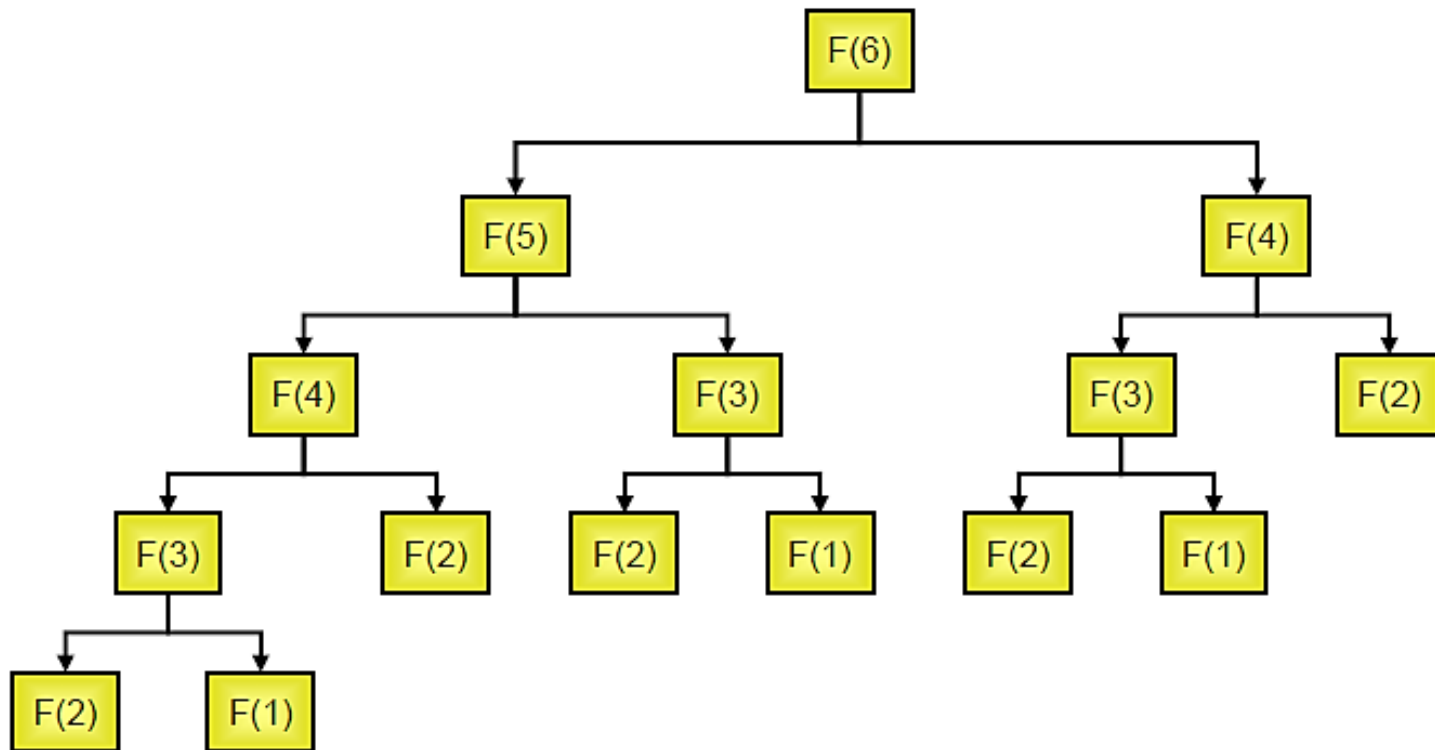
Ví dụ đệ quy nhị phân

```
#include <stdio.h>
```

```
int fibo(int n)
{
    if (n==0 || n==1)
    {
        return 1;
    }
    return fibo(n-1) + fibo(n-2);
}
```

```
int main()
{
    printf("\n 10 so Fibonacci dau tien la: ");
    for (int i=0; i<10; i++)
    {
        printf("%3d", fibo(i));
    }
}
```

Ví dụ đệ quy nhị phân



Đệ quy hồi tưởng

```
#include <stdio.h>
```

```
int X(int n)
```

```
{  
    if (n==0)  
        return 1;  
    return X(n-1) + Y(n-1);  
}
```

```
int Y(int n)
```

```
{  
    if (n==0)  
        return 1;  
    return n * n * X(n-1) + Y(n-1);  
}
```

```
int main()
```

```
{
```

```
printf("\n De quy ho tuong voi n=5");
```

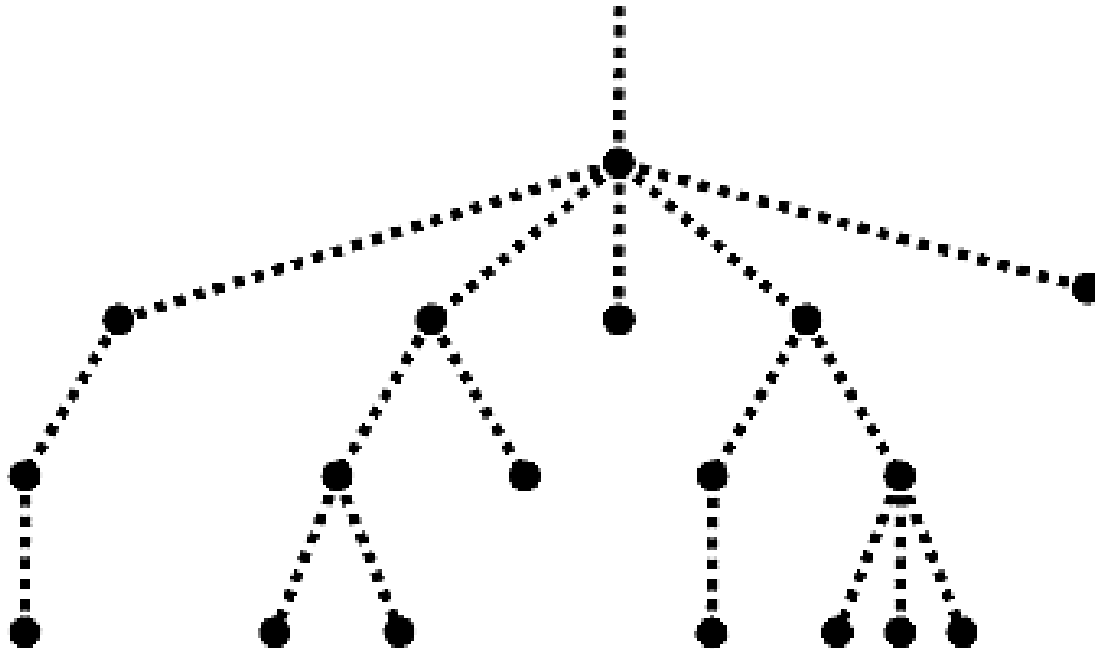
```
printf("\n X(5) = %d", X(5));
```

```
printf("\n Y(5) = %d", Y(5));
```

```
}
```

Đệ quy phi tuyến

- Trong thân hàm có lời gọi hàm lại chính nó được đặt bên trong vòng lặp.



Ví dụ đệ quy phi tuyến

```
#include <stdio.h>

/* Tính F(n)
n=0: F(n) = F(0) = 1
n>0: F(n) = 1^2 * F(0) + 2^2 * F(1) + ... + n^2
* F(n-1)*/

int F(int n)
{
    if (n==0)
        return 1;
    int f=0;
    for (int i=1; i<=n; i++)
    {
        f += i * i * F(i-1);
        printf("\n %d", f);
    }
    printf("---");

    return f;
}

int main()
{
    int n=5;
    printf("\n F(5) = %d", F(5));
}
```


Phân tích giải thuật đệ quy

- Phần dừng (base step): cần tạo ra một điều kiện dừng để xác định điểm kết thúc của bài toán, điểm dừng ở đây chính là xác định bài toán con nhỏ nhất trong bài toán lớn ban đầu.
- Phần đệ quy (recursion step): sẽ có phần gọi lại chính nó, và hàm khi gọi lại sẽ có số lượng và kiểu đối số giống hàm ban đầu chỉ khác về giá trị (thông thường scope sẽ nhỏ hơn).
- Ví dụ:

$$n! = \begin{cases} 1, n = 0 \\ n (n - 1)!, n > 0 \end{cases}$$

$$\text{Fib}(n) = \begin{cases} 1, n \leq 2 \\ \text{Fib}(n - 1) + \text{Fib}(n - 2), n > 2 \end{cases}$$

Đánh giá giải thuật đệ quy

- Ưu điểm
 - Khá đơn giản cho người lập trình khi suy nghĩ cách giải bài toán
 - Hàm đệ quy xây dựng đơn giản
 - Đệ quy là khái niệm dễ hiểu, dễ lập trình
- Hạn chế
 - Giải thuật viết theo phương pháp đệ quy khi thực thi chiếm bộ nhớ lớn

Khử đệ quy

- Mục đích
 - Tối ưu bộ nhớ
- Cách thực hiện
 - Sử dụng vòng lặp, biến tạm để khử đệ quy

Ví dụ 1 khử đệ quy bài toán số Fibonacci

```
#include<stdio.h>
```

```
int fib(int n)
```

```
{
```

```
    int f[n+2];
```

```
    int i;
```

```
    f[0] = 0;
```

```
    f[1] = 1;
```

```
    for (i = 2; i <= n; i++)
```

```
    {
```

```
        f[i] = f[i-1] + f[i-2];
```

```
    }
```

```
    return f[n];
```

```
}
```

```
int main ()
```

```
{
```

```
    int n = 9;
```

```
    printf("%d", fib(n));
```

```
}
```

Ví dụ 2 khử đệ quy bài toán số Fibonacci

```
#include<stdio.h>
```

```
int fib(int n)
{
    int a = 0, b = 1, c, i;
    if( n == 0)
        return a;
    for (i = 2; i <= n; i++)
    {
        c = a + b;
        a = b;
        b = c;
    }
    return b;
}
```

```
int main ()
{
    int n = 9;
    printf("%d", fib(n));
}
```



[This Photo](#) by Unknown Author is licensed under [CC BY-NC-ND](#)